Experiment-5

1. What is a Thread?

A thread is the smallest unit of execution within a process. A process may contain one or more threads. Threads of the same process share resources such as code, data, and files, but each thread has its own program counter, registers, and stack.

2. Process vs Thread

Process	Thread
A process is a program in execution.	A thread is a unit of execution within a process.
Has its own address space.	Shares the address space with other threads of the same process.
Process creation is relatively slower.	Thread creation is comparatively faster.
Context switching is more expensive.	Context switching is comparatively cheaper.
Processes communicate using IPC mechanisms.	Threads can communicate through shared memory.
A process has its own resources.	Threads share most resources of their process.
Failure of one process generally does not directly affect another process.	Failure of one thread can affect the entire process.
Requires more memory.	Requires less additional memory.

3. Types of Threads

There are mainly two types of threads:

1. User-Level Threads

User-level threads are managed by a user-level thread library instead of the operating system kernel. The operating system may treat the entire process as a single execution unit, while the thread library manages the individual threads.

Advantages:

- Faster to create.
- Faster thread switching.
- Less kernel involvement is required.

Disadvantages:

- If one thread performs a blocking system call, the entire process may be blocked.
- True parallel execution on multiple CPU cores may be difficult in some models.

2. Kernel-Level Threads

Kernel-level threads are managed directly by the operating system. The kernel maintains information about each thread and schedules them independently for execution.

Advantages:

- If one thread is blocked, other threads can continue executing.
- Threads can execute in parallel on multiple CPU cores.
- Provides better support for multiprocessor systems.

Disadvantages:

- Thread creation and management are slower than user-level threads.
- More overhead is involved because the kernel manages the threads.

4. Multiprocessing vs Multithreading

Multiprocessing	Multithreading
Uses multiple processes.	Uses multiple threads within a process.
Each process has a separate address space.	Threads share the same address space.
Communication requires IPC mechanisms.	Communication can use shared memory.
Process creation is slower.	Thread creation is faster.

Context switching has higher overhead.	Context switching has lower overhead.
Requires more memory.	Requires comparatively less memory.
Failure of one process usually does not affect other processes.	Failure of one thread can affect the entire process.
Provides stronger isolation.	Provides less isolation.
Suitable when tasks require isolation.	Suitable when tasks need to share data and resources.

Aim: Programs to illustrate threads. Implement thread functions.

Description: A thread is the smallest unit of execution within a process. A process can contain multiple threads that execute concurrently and share the resources of the same process. POSIX Threads, commonly known as pthread, provide a set of functions for creating, managing, and terminating threads.In this program, a new thread is created using pthread_create(). The created thread performs its assigned task and terminates using pthread_exit() after completing its execution. The main thread uses pthread_join() to wait for the child thread to complete before continuing its execution. This ensures that the main thread and child thread are properly synchronized and that the program completes in the correct order.

Algorithm:

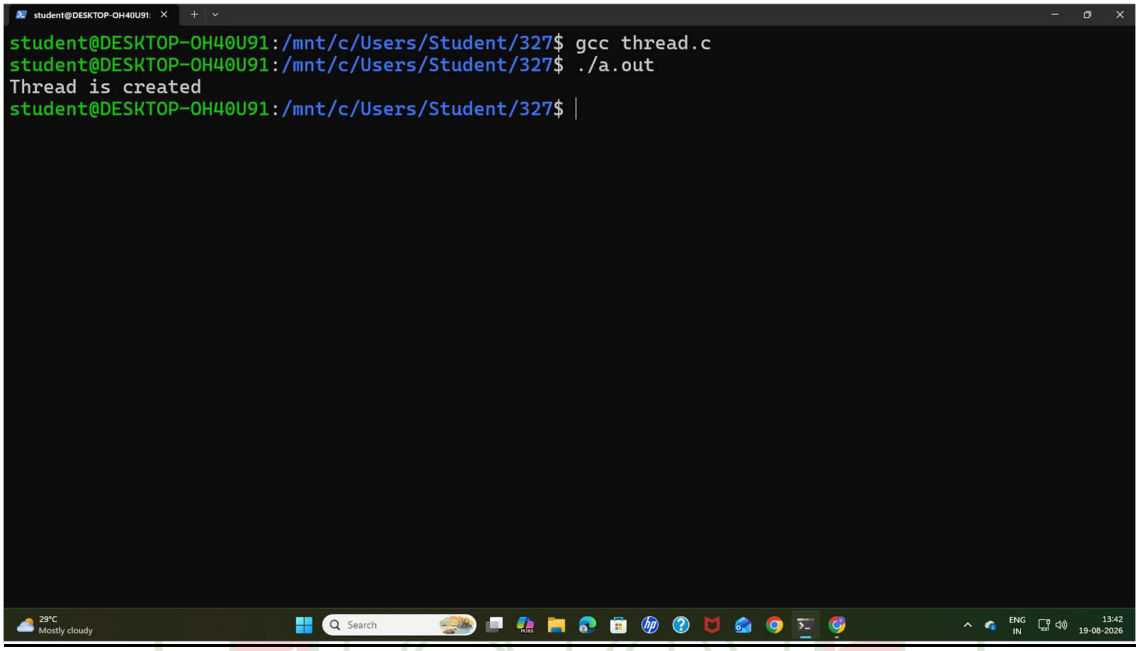
1. Start the execution of the program.
2. Define a thread routine function named Tcreate:
   - Step 2.1: Print the message "Thread is created\n" to the console.
   - Step 2.2: Return NULL to signify the successful completion and termination of the thread.
3. Begin the main function:
   - Step 3.1: Declare a thread variable t of type pthread_t to store the unique identifier for the new thread.
   - Step 3.2: Call the pthread_create() function to spawn a new concurrent thread.
     - Pass the memory address of t (&t).
     - Pass NULL for default thread attributes.
     - Pass the Tcreate function as the routine for the new thread to execute.
     - Pass NULL as the argument to the Tcreate function.
   - Step 3.3: Call the pthread_join() function on thread t. This blocks (pauses) the main program from moving forward until the newly created thread t has completely finished executing its Tcreate function.
   - Step 3.4: Once the thread has finished and joined, return 0 to exit the main program successfully.
4. Stop.

Source Code:

```
#include<stdio.h>
#include<pthread.h>
void* Tcreate(void* arg){
printf("Thread is created\n");
return NULL;
}
int main(){
pthread_t t;
pthread_create(&t, NULL,Tcreate,NULL);
pthread_join(t,NULL);
```

```
return 0;
}
```

Output:



Aim: WAP to create a thread to demonstrate sharing of values among two threads.
Description: A thread is the smallest unit of execution within a process. All threads belonging to the same process share the same memory space, allowing them to access and modify common variables without requiring Inter-Process Communication (IPC).In this program, a global variable named shared is initialized to zero. Two threads are created using the POSIX Threads (pthread) library. Both threads execute the same thread function, where they increment the shared variable and print its updated value.The program demonstrates how threads can share and access the same memory location. Any modification made to the shared variable by one thread can be observed by the other thread because both threads use the same process memory. The main thread uses pthread_join() to wait for both child threads to complete their execution. Finally, the program displays the updated value of the shared variable.

Algorithm:

1. Start the execution of the program.
2. Define the first thread function named create1:
- Step 2.1: Cast the passed argument arg into a character pointer a.
- Step 2.2: Print "Thread1 is created\n".
- Step 2.3: Append the string "Thread1 " to the character array a using strcat().
- Step 2.4: Terminate the thread using pthread_exit(NULL).
3. Define the second thread function named create2:
- Step 3.1: Cast the passed argument arg into a character pointer a.
- Step 3.2: Print "Thread2 is created\n".
- Step 3.3: Append the string "Thread2 " to the character array a using strcat().
- Step 3.4: Terminate the thread using pthread_exit(NULL).
4. Begin the main function:
- Step 4.1: Initialize a character array a with the string "a is modified from ".
- Step 4.2: Declare two thread variables, t1 and t2, of type pthread_t.

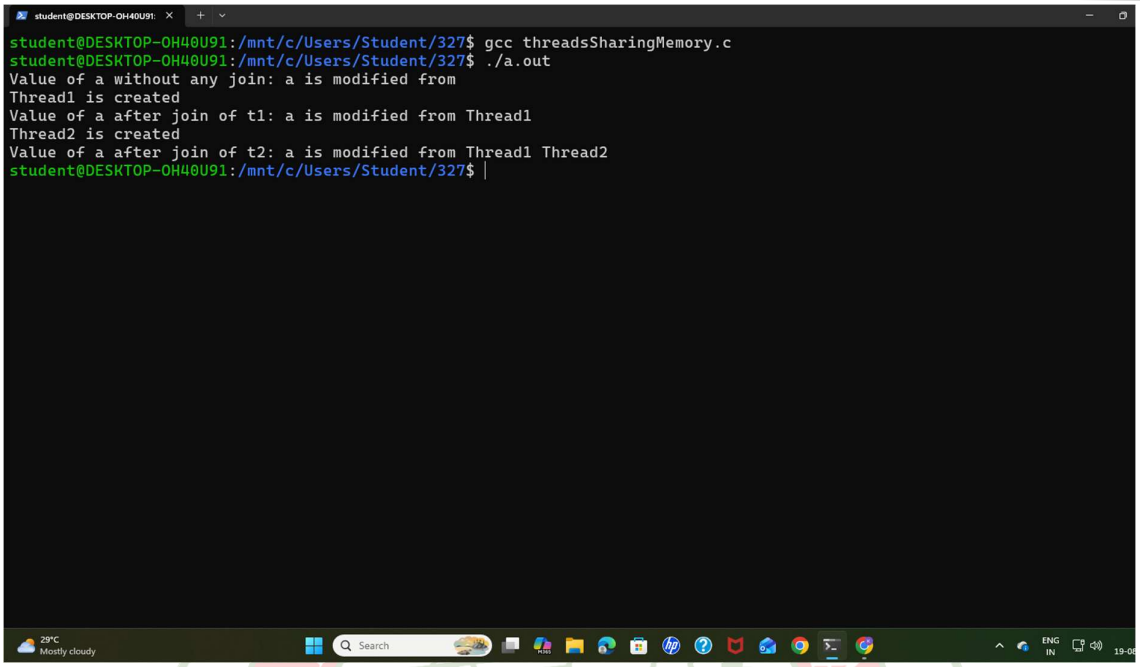
- Step 4.3: Print the initial value of a (before any thread modification).
- Step 4.4: Create thread t1 using pthread_create(), passing the array a as the argument to the create1 function.
- Step 4.5: Call pthread_join() on t1 to pause the main program until t1 finishes modifying the string and exits.
- Step 4.6: Print the updated value of a after t1 has joined.
- Step 4.7: Create thread t2 using pthread_create(), passing the array a as the argument to the create2 function.
- Step 4.8: Call pthread_join() on t2 to pause the main program until t2 finishes modifying the string and exits.
- Step 4.9: Print the final updated value of a after t2 has joined.
- Step 4.10: Return 0 to exit the main program.

5. Stop.

Code:

```
#include<stdio.h>
#include<pthread.h>
#include<string.h>
void* create1(void* arg){
char *a = (char *)arg;
printf("Thread1 is created\n");
strcat(a,"Thread1 ");
pthread_exit(NULL);
return NULL;
}
void* create2(void* arg){
char *a = (char *)arg;
printf("Thread2 is created\n");
strcat(a,"Thread2 ");
pthread_exit(NULL);
return NULL;
}
int main(){
char a[100] = "a is modified from ";
pthread_t t1,t2;
printf("Value of a without any join: %s\n", a);
pthread_create(&t1,NULL, create1,(void*)a);
pthread_join(t1,NULL);
printf("Value of a after join of t1: %s\n", a);
pthread_create(&t2,NULL, create2,(void*)a);
pthread_join(t2,NULL);
printf("Value of a after join of t2: %s\n", a);
return 0;
}
```

Output:



Aim: WAP to demonstrate producer-consumer using threads.
Description : The Producer-Consumer problem is a classical synchronization problem in Operating Systems. It involves a **producer**, which adds data to a shared buffer, and a **consumer**, which removes data from the buffer. In this program, both threads use a fixed-size circular buffer, where **in** indicates the next position for insertion and **out** indicates the next position for removal. If the buffer is full, the producer waits for space to become available, while the consumer waits when the buffer is empty. Thus, the program demonstrates proper synchronization and coordination between threads while accessing a shared resource.
Algorithm:

1. Start.
2. Initialize a shared circular buffer of size 5, along with in, out, and count variables set to 0.
3. Define the Producer thread function (loops 5 times):
   - Busy-wait (pause) if the buffer is full (count == SIZE).
   - Insert the current value into buffer[in].
   - Update the in index circularly and increment count.
   - Print the produced item and sleep for 1 second.
4. Define the Consumer thread function (loops 5 times):
   - Busy-wait (pause) if the buffer is empty (count == 0).
   - Read the value from buffer[out].
   - Update the out index circularly and decrement count.
   - Print the consumed item and sleep for 2 seconds.
5. In the main function, create the Producer and Consumer threads using pthread_create().
6. Wait for both threads to finish execution using pthread_join().
7. Stop.

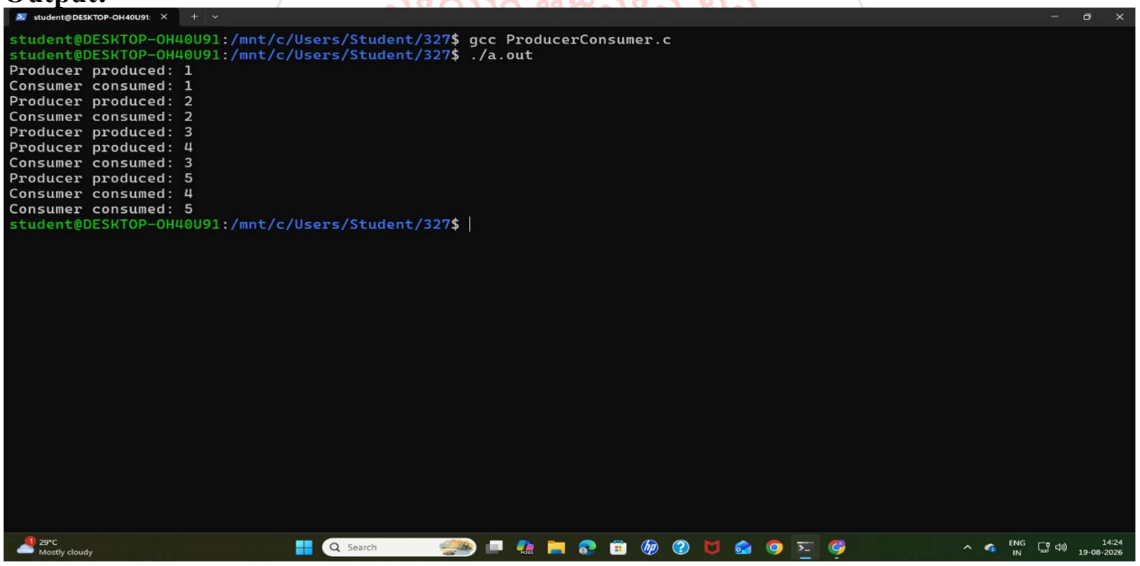
Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define SIZE 5

int buffer[SIZE];
int in = 0;
int out = 0;
int count = 0;
void *producer(void *arg){
    int item;
    for (int i = 1; i <= 5; i++){
        while (count == SIZE);
        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % SIZE;
        count++;
        sleep(1);
    }
    pthread_exit(NULL);
}
void *consumer(void *arg){
    int item;
    for (int i = 1; i <= 5; i++){
        while (count == 0);
        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % SIZE;
        count--;
        sleep(2);
    }
    pthread_exit(NULL);
}
int main(){
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

Output:



Aim: WAP to calculate sum of n numbers using threads.

Description : This program demonstrates the use of a POSIX thread to calculate the sum of the first n natural numbers. The program takes an integer n from the user and stores it in a global variable. A new thread is created using the pthread_create() function, which executes the thread_function() to calculate the sum using the formula Sum = n × (n + 1) / 2. The calculated result is displayed by the created thread. After completing the calculation, the thread terminates using pthread_exit().

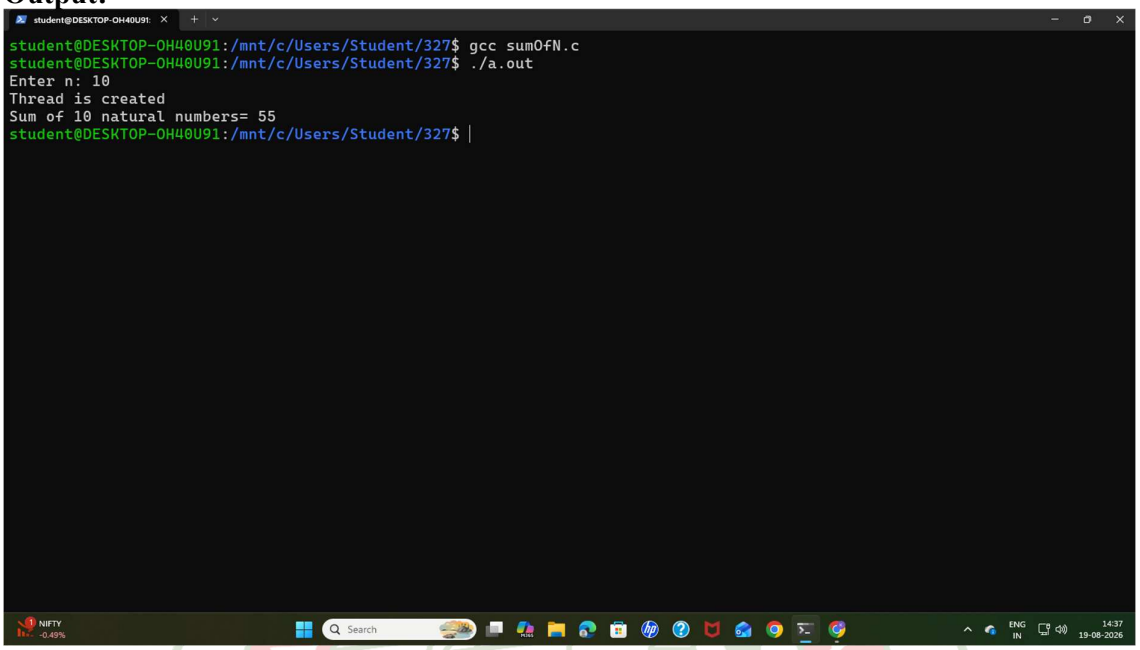
Algorithm:

1. Start the program.
2. Define the thread function (Tcreate):
   - Extract the integer n from the passed argument.
   - Print a message confirming the thread is created.
   - Use a loop to calculate the sum of natural numbers from 1 to n.
   - Print the calculated sum and exit the thread.
3. Begin main function:
   - Prompt the user and read an integer input n.
   - Create a thread (t) using pthread_create(), passing the address of n to the Tcreate function.
   - Wait for the thread to complete its task using pthread_join().
   - Return 0 to exit the main program.
4. Stop.

Code:

```
#include<stdio.h>
#include<pthread.h>
void* Tcreate(void* arg){
printf("Thread is created\n");
int n = *(int*) arg;
int sum=0;
for(int i=1; i<=n;i++){
sum += i;
}
printf("Sum of %d natural numbers= %d\n", n,sum);
return NULL;
}
int main(){
int n;
printf("Enter n: ");
scanf("%d" ,&n);
pthread_t t;
pthread_create(&t,NULL, Tcreate,(void*)&n);
pthread_join(t,NULL);
return 0;
}
```

Output:



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979